

Talksy — Initial Database Architecture (Scalable Foundation)

Goal

Build a scalable emotional companion app database structure.

This structure is designed for:

- onboarding flow
- auth + OTP
- emotional AI chat
- companion system
- subscriptions/paywall
- future analytics
- scalable conversation memory
- multi-language support

Recommended Core Tables

Phase 1 (Build NOW)

These are enough for:

```
Splash
→ Login
→ OTP
→ Onboarding
→ Chat
```

1. users

Main user table.

2. otp_sessions

OTP verification flow.

3. onboarding_profiles

Stores onboarding answers.

4. companions

AI companion master profiles.

5. user_companions

Which companion belongs to which user.

6. conversations

Conversation session grouping.

7. messages

Chat messages.



Recommended Final Architecture

```
users
otp_sessions
onboarding_profiles
companions
user_companions
conversations
messages
subscriptions
payments
usage_limits
user_preferences
user_memories
ai_personality_profiles
companion_voice_profiles
push_tokens
analytics_events
reports
```



PHASE 1 TABLE DESIGN

1 users

Purpose

Stores:

- mobile auth
 - account state
 - onboarding completion
-

Columns

Column	Type
id	uuid PK
phone_number	varchar unique
name	varchar nullable
is_verified	boolean
onboarding_completed	boolean
created_at	timestampz
updated_at	timestampz

2 otp_sessions

Purpose

Stores OTP verification attempts.

Columns

Column	Type
id	uuid PK
phone_number	varchar
otp_code	varchar
expires_at	timestampz
verified	boolean

Column	Type
attempts	integer
created_at	timestampz

3 onboarding_profiles

Purpose

Stores emotional onboarding answers.

Columns

Column	Type
id	uuid PK
user_id	uuid FK
language	varchar
companion_name	varchar
vibe	varchar
companion_type	varchar
personality_type	varchar
gender_preference	varchar
created_at	timestampz
updated_at	timestampz

4 companions

Purpose

Master companion templates.

Example:

- caring girlfriend
- funny best friend
- calm listener
- flirty companion

Columns

Column	Type
id	uuid PK
name	varchar
type	varchar
personality	varchar
avatar_url	text
prompt_template	text
active	boolean
created_at	timestampz

5 user_companions

Purpose

Maps users to companion instances.

Because later:

- users may switch companions
- multiple companions possible
- memory separation needed

Columns

Column	Type
id	uuid PK
user_id	uuid FK
companion_id	uuid FK
custom_name	varchar
active	boolean
created_at	timestampz

6 conversations

Purpose

Conversation session grouping.

One user can have:

- multiple chat sessions
- archived sessions
- future search/history

Columns

Column	Type
id	uuid PK
user_id	uuid FK
companion_id	uuid FK
title	varchar nullable
archived	boolean
created_at	timestampz
updated_at	timestampz

7 messages

Purpose

Stores all chat messages.

Columns

Column	Type
id	uuid PK
conversation_id	uuid FK
sender_type	varchar
message_type	varchar

Column	Type
content	text
ai_model	varchar nullable
created_at	timestampz

sender_type values

```
user
assistant
system
```

message_type values

```
text
image
voice
system
```

IMPORTANT ARCHITECTURE DECISIONS

Use UUID everywhere

Do NOT use incremental integer IDs.

Why:

- safer
- scalable
- better distributed systems
- easier future sync

Keep onboarding separate from users table

Do NOT dump onboarding columns into users.

Bad:

```
users.vibe
users.personality
users.language
```

Good:

```
onboarding_profiles
```

Much cleaner.

✓ Keep conversations separate from messages

Never store giant chat arrays in users.

Bad:

```
users.messages_json
```

Good:

```
conversations
messages
```

✓ Keep companion templates separate

This gives future flexibility:

- premium companions
 - dynamic prompts
 - companion marketplace
 - AI tuning
 - seasonal personalities
-

🚫 Things NOT to build yet

Avoid building these right now:

- vector memory DB
- embeddings pipeline
- complex analytics

- event sourcing
- realtime scaling infra
- multi-agent systems
- LangChain complexity

Early-stage apps die from overengineering.

Recommended Tech Pairing

Backend

Node.js + Express

Database

PostgreSQL

ORM / Query Layer

Choose ONE later:

Drizzle ORM
OR
Supabase client
OR
Prisma

Do NOT mix multiple systems.

Recommended Build Order

Step 1

users
otp_sessions

Step 2

onboarding_profiles

Step 3

```
companions  
user_companions
```

Step 4

```
conversations  
messages
```



Most Important Principle

Build for:

```
clarity first
```

NOT:

```
enterprise complexity first
```

Your current stage needs:

- fast iteration
- stable architecture
- easy debugging
- emotional UX
- quick feature shipping

NOT giant infra complexity.